



DATABASE MANAGEMENT SYSTEMS

UNIT-1

SHORT ANSWER QUESTIONS

1. Write any two differences between Relational and Non-Relational database.

Feature	Relational database	Non-Relational database
Data Structure	Stores data in tables	Stores data as documents or graph
Schema	Has a fixed schema	Uses a flexible / dynamic schema
Relationships	Supports relationships using foreign keys	Usually does not support foreign keys directly
Examples	MySQL, Oracle	MongoDB, Cassandra

2. Define the following.

a) Schema b) Instance

Schema: A schema is the blueprint or structure that defines how data is organized and stored in a database.

Instance: It is a collection of information that stored in a database at a particular moment is called an instance of the database.

3. Write applications of DBMS

Ans: Applications of DBMS:

- **Enterprise Information:** Accounting, Sales, Manufacturing
- **Banking and Finance:** customer information, accounts, loans, transactions, Credit card transactions
- **Universities and colleges:** student information, course registration, grades.
- **Library Management System.**
- **Railway Reservation System.**

4. Define Generalization of E-R Model

Ans: It is the process of **extracting common properties** from a set of entities and creating a generalized entity from it. It is a **bottom-up approach** in which two lower-level entities combine to form a **higher-level entity**.

5. Write about DBMS Components.

Ans: A Database Management System (DBMS) consists of five major components that enable efficient data management. They work together to process, store, and secure data.

The Major Components are:

1. Software
2. Hardware
3. Data
4. Procedures
5. Users

6. Define specialization in E-R Model.

A. In specialization, an entity is divided into sub-entities based on their characteristics. It is a top-down approach where higher level entity is specialized into two or more lower-level entities.

LONG ANSWER QUESTIONS

1. Write the Functions of DBMS detail.

Ans: A Database Management System (DBMS) is a software system that enables users to define, create, maintain, and control access to a database.

The main functions of a DBMS include:

- a) **Data Storage:** The DBMS is responsible for efficiently storing and organizing data on disk or other storage media.
- b) **Data Retrieval:** Users can query the database to retrieve specific information. The DBMS processes these queries and returns the requested data.
- c) **Data Manipulation:** This involves updating, inserting, and deleting data in the database. For example, adding a new employee record or updating an existing employee's salary.
- d) **Data Security:** The DBMS provides mechanisms to control access to the data, ensuring that only authorized users can view or modify it. This includes features like user authentication, access control lists, and encryption.
- e) **Data Integrity:** The DBMS implements rules to maintain the accuracy and consistency of the data.

- f) **Concurrency Control:** In multi-user environments, the DBMS manages simultaneous access to the data to prevent conflicts and ensure data consistency.
- g) **Backup and Recovery:** The DBMS provides tools for backing up data and restoring it in case of failures or data loss.

2. What is Data model. Explain the Relational model in detail.

- Data Models defines that how data is structured, stored, and manipulated within a database.
- Data models use a set of symbols and text to represent the information.

Relational model:

- The relational model represents how data is stored and managed in Relational Databases.
- In this model, the information is stored in the form of row and columns. The basic structure of a relational model is tables. So, the tables are also called relations in the relational model.

Relational Model Concepts in DBMS:

a) Tables: In the Relational model, relations are saved in the table format. It is stored along with its entities. A table has two properties rows and columns. Rows represent records and columns represent attributes.

b) Attribute: Attributes are the properties which define a relation. e.g., Student Rollno, Name, etc.

c) Tuple: It is nothing but a single row of a table, which contains a single record.

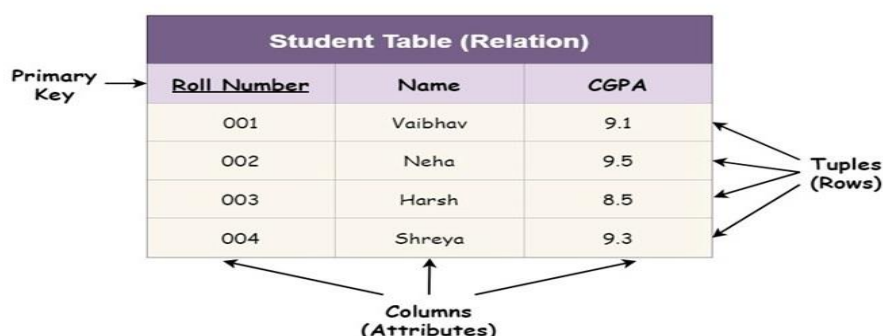
d) Degree: The total number of attributes which in the relation is called the degree of the relation.

e) Cardinality: Total number of rows present in the Table.

f) Attribute domain: Every attribute has some pre-defined value and scope which is known as attribute

domain.

Relational Model in DBMS



Advantages:

- **Simple:** This model is simpler compared to the network and hierarchical model.
- **Scalable:** This model can be easily scaled as we can add as many rows and columns we want.
- **Structural Independence:** We can make changes in database structure without changing the way to access the data.

Disadvantages:

- **Hardware Overheads:** For hiding the complexities and making things easier for the user this model requires more powerful hardware computers and data storage devices.
- **Inflexibility:** Changing schemas for evolving requirements can be time-consuming and disruptive.
- **Poor for Unstructured Data:** Not ideal for semi-structured or unstructured data (like JSON, XML) compared to NoSQL databases.

3. Write the difference between File System and DBMS.

Feature	File System	DBMS
Meaning	Stores data in files (text, binary, etc.)	Stores data in databases using tables
Data Organization	Data is stored in separate files	Data is stored in structured tables
Redundancy	High (same data may be repeated in many files)	Low (data is centralized and controlled)
Data Consistency	Poor	High
Data Security	Limited security	Strong security (authentication, authorization)
Data Integrity	Hard to enforce	Integrity constraints (keys, rules)
Concurrency	Not well supported	Multiple users can access safely
Data Sharing	Difficult	Easy and efficient

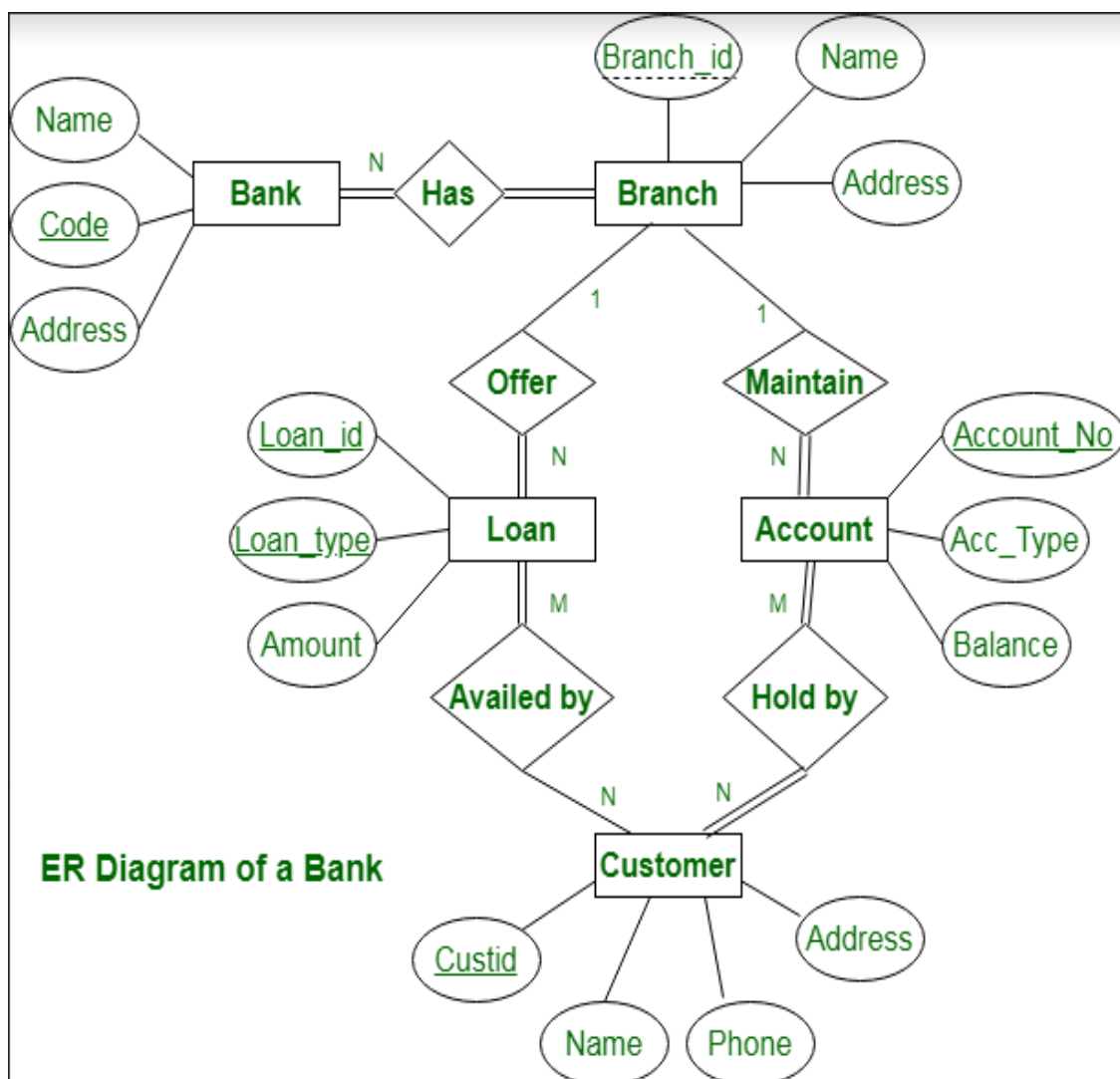
4. Draw an E-R diagram for any banking enterprise system.

An **Entity-Relationship (E-R) diagram** is a conceptual model used in DBMS to represent:

- **Entities** (objects)
- **Attributes** (properties)
- **Relationships** between entities

In a **banking enterprise system**, the E-R diagram shows how customers, accounts, loans, branches, and transactions are connected.

An E-R diagram for a banking enterprise system visually represents entities like Customer, Account, Branch, Loan, and Transaction along with their attributes and relationships, helping in effective database design and management.



5. Write the System Architecture of DBMS.

Ans: The high-level Database System Architecture is divided into four main layers:

- Users
- The Query Processor
- The Storage Manager
- Disk Storage.

a) User Layer (The Interface):

This top layer represents the different ways people interact with the database.

- Naive Users
- Application Programmers
- Sophisticated Users
- Database Administrator (DBA)

b) Query Processor Layer:

This layer is the "brain" of the database. It translates user requests into actions the computer can execute.

- DDL Interpreter
- DML Queries
- Compiler and Linker
- DML Compiler and Organizer
- Query Evaluation Engine

c) Storage Manager Layer:

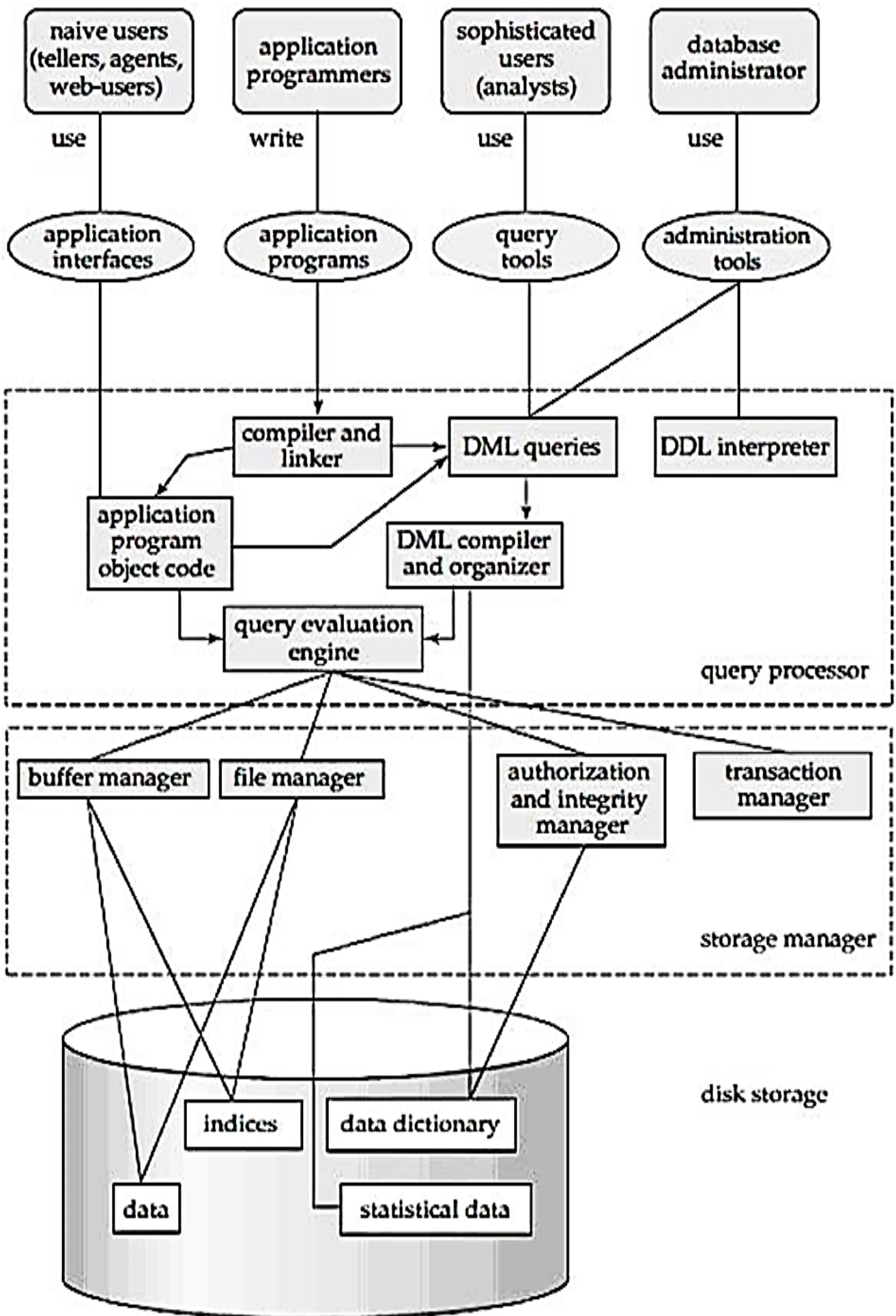
This layer acts as the bridge between the high-level queries and the raw data stored on the disk.

- Authorization and Integrity Manager
- Transaction Manager
- Buffer Manager
- File Manager

d) Disk Storage Layer:

This is where the actual bits and bytes live permanently on the physical hardware.

- Data
- Indices
- Data Dictionary
- Statistical Data



6. Write about Entity, Attributes and Relationships.

A) Entity:

Entity is a Thing or Object in the real world. It can be a person, place, or even a concept.

- An Entity is represented by A Rectangular symbol.

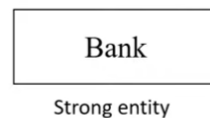
Example: Teachers, Students, Course, Building, Department, etc are some of the entities.

There are two main types of entities: 1. Strong Entity

2. Weak Entity

- **Strong Entity:** A Strong Entity is a type of entity that has a key Attribute that can uniquely identify and it does not depend on any other Entity.

Example:



- **Weak Entity:** A Weak Entity cannot be uniquely identified by its own attributes alone. It depends on a strong entity to be identified.

Example:

a bank account cannot be uniquely identified without knowing the bank to which the account belongs, so bank account is weak entity.



B) Attributes:

- An Attribute is a property that describes an entity in a database. It provides specific information used to identify, categorize, and manage entities effectively. Without attributes, entities have no meaningful data.
- Simply, Properties of an entity are called Attributes.

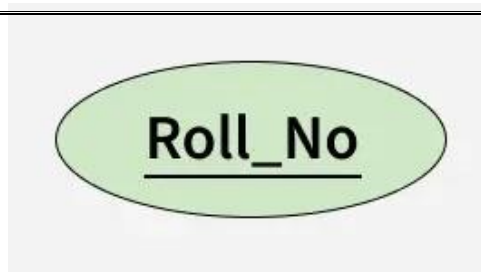
Example: Student is an entity and student has Roll_No, Name, DOB, Age..

- In ER diagram, the attribute is represented by an ellipse or an oval.

Types of Attributes and their Symbols:

a) Key Attribute:

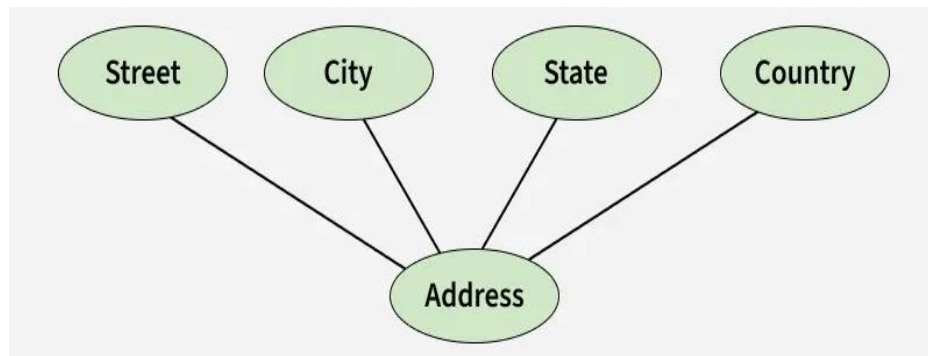
The attribute which uniquely identifies each entity in the entity set is called the key attribute. For example, Roll_No will be unique for each student.



In ER diagram, the **key attribute** is represented by an **oval with an underline**.

b) Composite Attribute

An attribute composed of many other attributes is called a composite attribute. For example, the Address attribute of the student Entity type consists of Street, City, State, and Country.



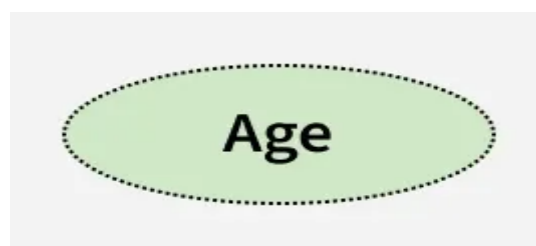
c) Multivalued Attribute

An attribute consisting of more than one value for a given entity. For example, Phone_No (can be more than one for a given student). In ER diagram, a **multivalued attribute** is represented by a **double oval**.



d) Derived Attribute

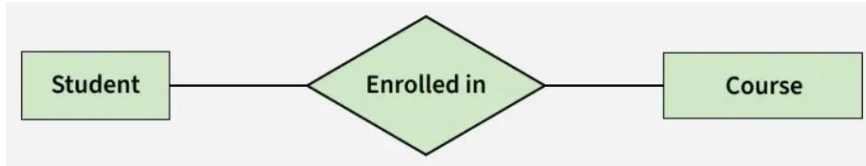
An attribute that can be derived from other attributes of the entity type is known as a derived attribute. e.g.; Age (**can be derived from DOB**). In ER diagram, the **derived attribute** is represented by a **dashed oval**.



C) Relationships:

A Relationship represents the association between entity types or Relationship gives the relationship between entities.

For example, 'Enrolled in' is a relationship type that exists between entity type Student and Course. In ER diagram, the relationship type is represented by a diamond and connecting the entities with lines.



Degree of Relationship: The number of different entity sets participating in a relationship set is called the degree of a relationship set.

1. Unary/Recursive Relationship
2. Binary Relationship
3. Ternary Relationship
4. N-ary Relationship

Cardinality in ER Model:

The maximum number of times an entity of an entity set participates in a relationship set is known as cardinality.

Cardinality can be of different types:

- 1.One-to-one:** One entity from a entity set can be associated with at most one entity of other entity set.
- 2.One-to-many:** One entity from A entity set can be associated with more than one entity of any other entity set.
- 3.Many-to-one:** More than one entity from a entity set can be associated with at most one entity of any other entity set.
- 4.Many-to-many:** When entities in all entity sets can take part more than once in the relationship.

UNIT II

SHORT ANSWER QUESTIONS

1) Write the difference between DDL and DML commands.

- **DDL** is a Data Definition Language that is **used to define data** structures where as **DML** is a Data Manipulation Language that is **used to manipulate data** itself.

Example:

DDL commands are creating a table, and altering a table are instructions in SQL.

DML commands are insert, update, and delete are instructions in SQL.

2) What are set theory operations in SQL?

- Set theory operations in database management systems (DBMS) are **used to manipulate & query data across multiple tables**.
- Set theory operations are fundamental in relational algebra and SQL, allowing users to perform various operations on sets of data.

Example:

UNION , INTERSECTION, DIFFERENCE..etc

3) What is a constraint in SQL?

- Constraints are a set of rules. It is used to maintain the quality of information. Constraint is used to guard against accidental damage to the database.
- Constraints ensure that the data insertion, updating, and other processes have to be performed in such a way that data integrity is not affected.

SQL Constraints

1. not null
2. auto increment
3. unique
4. primary key

4) What is a PRIMARY KEY? State one property.

Definition:

A PRIMARY KEY is a constraint in DBMS used to uniquely identify each record (row) in a table.

Property:

- It does not allow duplicate values.
- It cannot contain NULL values.)

Example:

```
CREATE TABLE Student (  
  
    Student_ID INT PRIMARY KEY,  
    Name VARCHAR(50)  
  
);
```

5) What is a FOREIGN KEY? Why is it used?

A Foreign Key is used to link one table with another table. It references the Primary Key of another table.

Use of Foreign Key:

- Data Integrity
- Query Optimization
- Establishing Relationships
- Data Security
- Database Maintenance.

6) Write the syntax of the WHERE clause.

The "WHERE" clause is used to filter the result set based on specific conditions. It allows you to retrieve only the rows that satisfy the given condition(s).

Syntax:

```
SELECT column1, column2, ... FROM table  
WHERE condition;
```

LONG ANSWER QUESTIONS**✓1) Explain Primary Key and Foreign Key with examples and advantages.****Primary Key::**

A Primary Key uniquely identifies each record in a table. It cannot contain NULL or duplicate values.

Example:

```
CREATE TABLE Student (  
  
    Student_ID INT PRIMARY KEY,  
    Name VARCHAR(50)  
  
);
```

Explanation:

In above example code after creating the database, we are creating a table with name Student. **Student_ID** is a column of type INT that stores integer values. **PRIMARY KEY** uniquely identifies each record in the table and does not allow duplicate or NULL values. **Name** is a column of type VARCHAR(50) that can store character strings up to 50 characters in length.

Advantages:

- Ensures unique identification
- Improves data integrity
- Helps establish relationship

Foreign Key::

A Foreign Key is used to link one table with another table. It references the Primary Key of another table.

Example:

```
CREATE TABLE Enrollment (  
    Enrollment_ID INT,  
    Student_ID INT,  
    FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID)  
);
```

Explanation:

- Enrollment_ID is an integer column used to store the enrollment number.
- Student_ID is an integer column that stores the ID of a student.
- FOREIGN KEY is a constraint used to establish a relationship between two tables.
- Student_ID in the **Enrollment** table references Student_ID in the **Student** table.

Advantages:

- Maintains referential integrity
- Prevents invalid data entry
- Establishes table relationships

Note : Primary and Foreign keys help maintain database consistency and relationships.

2) Explain the SELECT statement using where clause with syntax.

The SELECT statement is used in DBMS to retrieve data from database tables. It allows the user to specify which column to display and which row to fetch based on conditions.

The Where clause is used along with SELECT filter records based on the specific condition. Only those rows that satisfy the condition in the WHERE clause are displayed in the result.

Syntax:

```
SELECT column_name  
FROM table_name  
WHERE condition
```

Important Clauses

FROM – Specifies table name

WHERE – Filters records

a) SELECT Clause

It Specifies columns to retrieve and * selects all columns

EXAMPLE:

```
CREATE TABLE employee(  
    emp_id int,  
    name varchar(50),  
    age int,  
    salary int  
);  
  
insert into employee(emp_id,name,age,salary)  
values(1,"Rani",25,20000),(2,"Sita",32,25000),(3,"Raju",30,22000),(4,"Laxman",26,15000);  
select * from employee;
```

Output:

	emp_id	name	age	salary
▶	1	Rani	25	20000
	2	Sita	32	25000
	3	Raju	30	22000
	4	Laxman	26	15000

In above example we have created a table with name employee and we get all the table data using Select Statement.

WHERE Clause

It is used to Filter rows based on conditions

```
select * from employee where emp_id=2;
```

Output:

	emp_id	name	age	salary
►	2	Sita	32	25000

In the above example we retrieve the information from employee table where emp_id=2 as shown in above table.

3) Describe table creation and modification in SQL using CREATE and ALTER commands.

CREATE TABLE is used to create a new table with specified columns and data types.

ALTER TABLE is used to modify the structure of an existing table.

CREATE TABLE Syntax

```
CREATE TABLE Table_name (  
    Column1 type,  
    Column2 type,  
    Column3 type  
);
```

- If we want to Store numbers give type as int
- If we want to store names give type as varchar(range)
- If we want to store decimals give type as Decimal(range before and after decimal)

Example:

```
CREATE database Students;  
use Students;  
CREATE table classA(  
    rno int,name varchar(50),age int,marks int);  
insert into classA values(1,"Ram",18,98),  
(2,"Sita",18,96),(3,"Laxman",17,95);  
select * from classA;
```

Output:

	rno	name	age	marks
	1	Ram	18	98
	2	Sita	18	96
	3	Laxman	17	95

In above table we created a table with name classA and with columns rno,name,age,marks.

ALTER TABLE Syntax

1.Add Column:

To add new columns to old table we use the below **syntax**.

```
ALTER TABLE Table_name  
ADD new_value type;
```

Example:

```
Alter table classA  
add grade varchar(10);  
select * from classA;
```

Output:

	rno	name	age	marks	grade
	1	Ram	18	98	NULL
	2	Sita	18	96	NULL
	3	Laxman	17	95	NULL

From above table we can observe that a new grade column is added to ClassA table.

2.Modify Column

The MODIFY clause is used to change the datatype or size of an existing column in a table.

Syntax:

```
ALTER TABLE Table_name  
MODIFY column_name New_datatype;
```

Example:

```
Alter table classA  
modify marks decimal(10,2);  
insert into classA values(5,"Varun",26,93.5,"B");  
select * from classA;
```

Output:

rno	name	age	marks	grade
1	Ram	18	98.00	NULL
2	Sita	18	96.00	NULL
3	Laxman	17	95.00	NULL
5	Varun	26	93.50	B

3.Drop Column

To delete column from existing table we use the below syntax.

Syntax:

```
ALTER TABLE Table_name  
DROP COLUMN column_name;
```

Example:

```
Alter table classA  
DROP COLUMN age;
```

Output:

rno	name	marks	grade
1	Ram	98.00	NULL
2	Sita	96.00	NULL
3	Laxman	95.00	NULL
5	Varun	93.50	B

From the above output,we can observe that age column is removed.

4) Write about SQL constraints in detail about any four with suitable examples.

Constraints are rules enforced by MySQL to maintain data integrity, accuracy, and consistency in a database.

The SQL constraints are

1. PRIMARY KEY
2. FOREIGN KEY:
3. AUTO_INCREMENT
4. NOT NULL
5. UNIQUE
6. DEFAULT
7. CHECK
8. UNSIGNED

1.PRIMARY KEY:

A Primary Key is a column (or a combination of columns) that uniquely identifies each row in a table.

- Uniquely identifies each row
- Cannot be NULL
- Cannot be duplicated
- Only one PRIMARY KEY per table

Syntax:

PRIMARY KEY (column_name)

Example Table:

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(50)  
);
```

Successful INSERT:

INSERT INTO students VALUES (1, 'Ravi');

Failed INSERT (duplicate key):

INSERT INTO students VALUES (1, 'Kiran');

Error:

ERROR 1062 (23000): Duplicate entry '1' for key 'students.PRIMARY'

2.FOREIGN KEY:

A Foreign Key is a column in a table that is a Primary Key in another table. It creates a relationship between two tables.

- Maintains relationship between tables
- Ensures child data exists in parent table

Syntax:

FOREIGN KEY (column) REFERENCES parent_table(column)

Example Tables:

```
CREATE TABLE departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);  
CREATE TABLE staff (  
    staff_id INT PRIMARY KEY,  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
);
```

Successful INSERT:

INSERT INTO departments VALUES (1, 'HR');

INSERT INTO staff VALUES (101, 1);

Failed INSERT (no parent record):

INSERT INTO staff VALUES (102, 5);

Error:

ERROR 1452 (23000): Cannot add or update a child row:

a foreign key constraint fails

3.AUTO_INCREMENT (used with PRIMARY KEY):

- Automatically generates unique numeric values
- When you don't want to manually insert IDs.

- Must be PRIMARY KEY or UNIQUE and Usually used for IDs

Syntax:

AUTO_INCREMENT

Example Table:

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY AUTO_INCREMENT,  
    product_name VARCHAR(50)  
);
```

Successful INSERT:

INSERT INTO orders (product_name) VALUES ('Laptop');

(MySQL auto-generates order_id)

Failed INSERT (manual duplicate ID):

INSERT INTO orders VALUES (1, 'Mobile');

4.NOT NULL:

- When data is mandatory
- Forces data to be entered
- Prevents missing critical information.

Syntax:

column_name datatype NOT NULL

Example Table:

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50) NOT NULL  
);
```

Successful INSERT:

INSERT INTO employees VALUES (1, 'Anita');

Failed INSERT:

INSERT INTO employees VALUES (2, NULL);

Error:

ERROR 1048 (23000): Column 'emp_name' cannot be null

5/ Discuss set theory operations in SQL with suitable examples.

Set theory operations in SQL are used to combine results from multiple SELECT queries. These operations follow mathematical set concepts and help retrieve data from two or more tables efficiently.

Types of Set Operations

1. UNION
2. UNION ALL
3. INTERSECT
4. EXCEPT

In the below code we create two tables with names Student and Teacher to understand the Set theory operations.

Rules

- Both SELECT statements must contain the same number of columns.
- Data types of columns must be compatible.
- Column order should be similar.

```
CREATE database Students;
use Students;
CREATE table classA(
rno int,name varchar(50),age int,marks int);
insert into classA values(1,"Ram",18,98),
(2,"Sita",18,96),(3,"Laxman",17,95);
select * from classA;
CREATE table classB(
id int,name varchar(50),age int,marks int);
insert into classB values(1,"Sita",17,85),
(2,"Vishnu",19,93),(3,"Divya",20,90);
select * from classB;
```

Output:

rno	name	age	marks
1	Ram	18	98
2	Sita	18	96
3	Laxman	17	95

id	name	age	marks
1	Sita	17	85
2	Vishnu	19	93
3	Divya	20	90

1. UNION

- Combines results of two queries and removes duplicate records from both tables.

```
SELECT name FROM classA
UNION
SELECT name FROM classB;
```

- Returns unique names from both tables.
- As Sita is present in 2 tables it returns only one time as shown below.

	name
	Ram
	Sita
	Laxman
	Vishnu
	Divya

2. UNION ALL

It Combines results including duplicate values from both tables.

```
SELECT name FROM classA
UNION ALL
SELECT name FROM classB;
```

Returns all names including repeated values as shown in below figure.(Sita repeated for 2 times)

name
Ram
Sita
Laxman
Sita
Vishnu
Divya

3. INTERSECT

- Returns records that are common in both queries.

```
SELECT age FROM classA
INTERSECT
SELECT age FROM classB;
```

Output:

age
17

As age 17 is present in both tables in common, it returns once as shown above.

4. EXCEPT (MINUS)

- Returns records from the first query that are not present in the second query.

```
SELECT name FROM classA
EXCEPT
SELECT name FROM classB;
```

Output

name
Ram
Laxman

- Returns students who are not sports students.

6) Describe Data Manipulation Operations in SQL (INSERT, UPDATE, DELETE) with Examples

Data Manipulation Language (DML) commands are used to **insert, modify, and delete records** in database tables. These commands directly affect the data stored in tables but do not change table structure.

1. INSERT Operation

Definition:

The INSERT command is used to add new records (rows) into a table. It allows users to store new data into the database and can insert single or multiple records.

Multiple rows can be inserted at a time by separating values with commas.

Syntax:

```
INSERT INTO Table_name (column1,column2, column3)
VALUES (value1, value2, value3);
```

Example:

```
CREATE database Students;
use Students;
CREATE table classA(
rno int,name varchar(50),age int,marks int);
insert into classA values(1,"Ram",18,98),
(2,"Sita",18,96),(3,"Laxman",17,95);
select * from classA;
```

Output:

rno	name	age	marks
1	Ram	18	98
2	Sita	18	96
3	Laxman	17	95

- INSERT INTO classA specifies the table into which data is inserted.
- VALUES keyword is used to provide the actual data.
- Each set of values inside parentheses represents **one row**.
- The values are inserted in the **same order as the columns** defined in the table:

2. UPDATE Operation

Definition:

The UPDATE command is used to modify existing records in a table. It allows updating one or more columns based on a specific condition.

Syntax:

```
UPDATE Table_name
SET column name = new_value
WHERE condition;
```

Example:

```
update classA set name="Vidya" where rno=1;
```

Output:

rno	name	age	marks
1	Vidya	25	98
2	Sita	32	96
3	Laxman	26	95

In the above table we can observe that at **rno=1**, the name is updated as **vidya**

3. DELETE Operation

Definition:

The DELETE command is used to remove one or more records from a table permanently based on specified conditions.

Syntax:

DELETE FROM Tablename

WHERE condition;

Example:

```
DELETE from classA where rno=2;
```

Output:

rno	name	age	marks
1	Vidya	25	98
3	Laxman	26	95

This query removes the student record with ID 2 from the table as shown in above figure.

UNIT 3

SHORT ANSWER QUESTIONS

1) Explain the BETWEEN operator with suitable example.

The BETWEEN operator is used to check whether a value lies within a specified range, including boundary values.

Example:

```
create table Student1(id int,name varchar(50),age int,dept varchar(50));
insert into Student1 values(1,"Deepu",19,"CSE"),
(2,"Nani",21,"ECE"),
(3,"Rani",20,"ECE"),
(4,"Joseph",17,"CSE"),
(5,"Madhu",18,"CSE");
select * from Student1;
select * from Student1 where age between 18 and 20;
```

Output:

id	name	age	dept
1	Deepu	19	CSE
3	Rani	20	ECE
5	Madhu	18	CSE

This query returns students whose age is between 18 and 20.

2) Discuss the Operator Precedence in SQL.

Operator precedence in SQL determines the order in which operators are evaluated in a query when multiple operators are used together. Operators with higher precedence are evaluated first.

Order of Operator Precedence

1. Parentheses ()- Force explicit grouping
2. Unary operators: +, - (bitwise NOT)
3. Multiplication *, Division /, Modulo %
4. Addition +, Subtraction -, Concatenation ||
5. Comparison operators
6. Logical Operators
7. Assignment = (in SET clauses or variable assignment)

1. Parentheses ()
2. Multiplication, Division, Modulus
3. Addition, Subtraction
4. Comparison
5. Logical Operators
6. Assignment Operator

3) Explain The IN operator in SQL with suitable Example.

IN Operator:

The IN operator is used to check whether a value matches any value in a specified list.

Example:

```
select * from Student1 where age in (18,20) ;
```

Output:

id	name	age	dept
3	Rani	20	ECE
5	Madhu	18	CSE

This query returns students whose age is in 18 or 20.

LONG ANSWER QUESTIONS

1Q: Logical Operators with Definitions and Examples

Logical Operators:

Logical operators are used in SQL WHERE clauses to combine multiple conditions. They return TRUE or FALSE depending on the logic.

The logical operators are

1. AND Operator
2. OR Operator
3. NOT Operator

To understand the logical operators, first create a table with id, name, age, dept columns as shown in below.

```
create table Student1(id int,name varchar(50),age int,dept varchar(50));
insert into Student1 values(1,"Deepu",19,"CSE"),
(2,"Nani",21,"ECE"),
(3,"Rani",20,"ECE"),
(4,"Joseph",17,"CSE"),
(5,"Madhu",18,"CSE");
select * from Student1;
```

Output:

id	name	age	dept
1	Deepu	19	CSE
2	Nani	21	ECE
3	Rani	20	ECE
4	Joseph	17	CSE
5	Madhu	18	CSE

a) AND Operator

Definition:

The AND operator is used to combine two or more conditions and returns results only when all conditions are true

Syntax:

SELECT * FROM Tablename

WHERE Condition1 AND Condition2;

Example:

```
select * from Student1 where age>18 AND dept="CSE";
```

Output:

	id	name	age	dept
	1	Deepu	19	CSE

The query returns the values whose age is greater than 19 and belongs to CSE

b) OR Operator

Definition:

The OR operator is used to combine multiple conditions and returns results when at least one condition is true.

Syntax:

```
SELECT * FROM Table_name
```

```
WHERE Condition1 OR Condition2;
```

Example:

```
select * from Student1 where age>20 OR dept="CSE";
```

Output:

	id	name	age	dept
	1	Deepu	19	CSE
	2	Nani	21	ECE
	4	Joseph	17	CSE
	5	Madhu	18	CSE

This query returns students whose age is greater than 20 or belong to CSE .

c)NOT Operator

Definition:

The NOT operator is used to reverse a condition and returns records that do not satisfy the given condition.

Stntax:

```
SELECT * FROM Table_name
```

```
WHERE NOT Condition;
```

Example:

```
select * from Student1 where NOT dept="CSE";
```

Output:

	id	name	age	dept
	2	Nani	21	ECE
	3	Rani	20	ECE

This query returns students who do not belong to the CSE course.

2) Explain Comparison operators in DBMS with Examples

Comparison operators are used to compare values in SQL queries and filter data.

Types of Comparison Operators

There are many types of comparison operators are there. they are

1. Equal (=)
2. Not Equal (!= or <>)
3. Greater Than (>)
4. Less Than (<)
5. Greater Than or Equal To (>=)
6. Less Than or Equal To (<=)

To understand the Comparison operators, first create a table with id, name, age, dept columns as shown in below.

```
create table Student1(id int,name varchar(50),age int,dept varchar(50));
insert into Student1 values(1,"Deepu",19,"CSE"),
(2,"Nani",21,"ECE"),
(3,"Rani",20,"ECE"),
(4,"Joseph",17,"CSE"),
(5,"Madhu",18,"CSE");
select * from Student1;
```

Output:

id	name	age	dept
1	Deepu	19	CSE
2	Nani	21	ECE
3	Rani	20	ECE
4	Joseph	17	CSE
5	Madhu	18	CSE

1. Equal (=)

It Checks whether values are equal.

Syntax:

```
SELECT * FROM Table_name  
WHERE Column_name = value;
```

Example:

```
select * from Student1 where dept="ECE";
```

Output:

id	name	age	dept
2	Nani	21	ECE
3	Rani	20	ECE

The above query returns whose dept is ECE as shown in above output

2. Not Equal (!= or <>)

It Checks whether values are not equal.

Syntax:

```
SELECT * FROM Table_name  
WHERE Column_name != value;
```

Example:

```
select * from Student1 where age!=20;
```

Output:

id	name	age	dept
1	Deepu	19	CSE
2	Nani	21	ECE
4	Joseph	17	CSE
5	Madhu	18	CSE

The above query returns the table by removing the row whose age is 20 as shown in above output

3. Greater Than (>)

It Returns values greater than specified value.

Syntax:

```
SELECT * FROM Table_name  
WHERE Column_name > value;
```

Example:

```
select * from Student1 where age>19;
```

Output:

id	name	age	dept
2	Nani	21	ECE
3	Rani	20	ECE

The above query returns whose age is greater than 19 as shown in above output

4. Less Than (<)

It Returns values less than specified value.

Syntax:

```
SELECT * FROM Table_name  
WHERE Column_name <value;
```

Example:

```
select * from Student1 where age<19;
```

Output:

id	name	age	dept
4	Joseph	17	CSE
5	Madhu	18	CSE

The above query returns whose age is less than 19 as shown in above output

4. Greater Than or Equal To (>=)

It Returns values greater than or equal to specified value.

Syntax:

```
SELECT * FROM Table_name  
WHERE Column_name >= value;
```

Example:

```
select * from Student1 where age>=19;
```

Output:

id	name	age	dept
1	Deepu	19	CSE
2	Nani	21	ECE
3	Rani	20	ECE

The above query returns whose age is greater than 19 as shown in above output

5. Less Than or Equal To (<=)

It Returns values less than or equal to specified value.

Syntax:

```
SELECT * FROM Table_name  
WHERE Column_name <= value;
```

Example:

```
select * from Student1 where age<=19;
```

Output:

id	name	age	dept
1	Deepu	19	CSE
4	Joseph	17	CSE
5	Madhu	18	CSE

The above query returns whose age is less than 19 as shown in above output

3) Explain Pattern Matching using LIKE operator in DBMS.

The SQL LIKE operator is used to search for a specific pattern within a column's text data. It works with wildcard characters to match partial strings, making it useful for flexible filtering.

- % represents any sequence of characters

To understand the Pattern matching using LIKE operators, first create a table with id, name, age, dept columns as shown in below.

```
create table Student1(id int,name varchar(50),age int,dept varchar(50));  
insert into Student1 values(1,"Deepu",19,"CSE"),  
(2,"Nani",21,"ECE"),  
(3,"Rani",20,"ECE"),  
(4,"Joseph",17,"CSE"),  
(5,"Madhu",18,"CSE");  
select * from Student1;
```

Output:

id	name	age	dept
1	Deepu	19	CSE
2	Nani	21	ECE
3	Rani	20	ECE
4	Joseph	17	CSE
5	Madhu	18	CSE

Syntax

SELECT column_name

FROM table_name

WHERE column_name LIKE pattern;

Example 1:

In this we write a query to return whose name is end with “u”.For that we use the syntax “%u” as shown in below example.

```
select * from Student1 where name like "%u";
```

Output:

id	name	age	dept
1	Deepu	19	CSE
5	Madhu	18	CSE

In above output we observe that the two names **Deepu** and **Madhu** are end with “u”

Example 2:

In this we write a query to return whose name is starts with “n”.For that we use the syntax “%n” as shown in below example.

```
select * from Student1 where name like "n%";
```

Output:

id	name	age	dept
2	Nani	21	ECE



DATABASE MANAGEMENT SYSTEMS
OBJECTIVE QUESTION BANK(MID-1)

1. DBMS manages the interaction between_ and database. []
A) Users B) Clients C) End users D) Stake holders
2. Which of the following is not involved in DBMS? []
A) Data B) End users C) Application Request D) HTML
3. Relational Algebra is []
A) Data Definition Language B) Meta language C) Procedural Query Language D) None of above
4. Key used to represent relationship between tables is called_____ []
A) Primary Key B) Foreign Key C) Super Key D) None of above
5. DBMS scalability mean_____ []
A) Fixed size B) No growth C) Ability to handle growth D) File conversion
6. What does an ER diagram mainly represent? []
A) Program logic B) Database queries C) Data objects and relationships D) Operating system
7. Which SQL clause is mainly used with comparison operators? []
A) Select B) From C) Where D) Order By
8. Which operator is used for pattern matching? []
A) In B) Like C) Between D) =
9. What does an RDBMS consist of? []
A) Collection Of Records B) Collection Of Keys C) Collection Of Tables D) Collection Of Fields
10. Which of the following is a component of the DBMS? []
A) Data B) Data Languages C) Data Manager D) All of the above
11. Data independence allows changes in schema without affecting _____. []
A) Hardware B) Database C) Application programs D) Users
12. A data model describes the structure and _____ of data. []
A) Processing B) Operations C) Relationships D) Encryption
13. Which is a conceptual data model? []
A) Relational model B) Network model C) ER model D) Physical model
14. A database schema represents the _____ of the database. []
A) Data values B) Structure C) Backup D) Users

15. A database instance represents the _____ state of the database. []
A) Future B) Previous ☒ C) Current D) Design
16. DBMS manages the interaction between _____ and database. []
☒ A) Users B) Clients C) End users D) Stake holders
17. A database schema represents the _____ view of the database. []
A) Physical ☒ B) Logical C) External D) User
18. The constraint that restricts attribute values to a specific range is called _____. []
A) Key constraint ☒ B) Domain constraint C) Referential constraint D) Entity constraint
19. The SELECT operation in relational algebra is used to select _____. []
A) Columns ☒ B) Rows C) Tables D) Databases
20. SELECT operation is a _____ operation. []
A) Binary B) Join ☒ C) Unary D) Set
21. DBMS manages the interaction between _____ and database. []
☒ A) Users B) Clients C) End users D) Stake holders
22. A database is a collection of _____ data. []
A) Unrelated B) Random ☒ C) Related D) Temporary
23. DBMS stands for _____ Management System. []
A) Data ☒ B) Database C) Document D) Digital
24. Which DBMS type stores data in table format? []
A) Hierarchical B) Network ☒ C) Relational D) Object-based
25. Non-relational databases are also called _____ databases. []
A) Structured B) Relational C) Flat-file ☒ D) NoSQL
26. The SQL command used to delete records from a table is _____. []
A) DROP B) REMOVE ☒ C) DELETE D) CLEAR
27. A primary key must contain _____ values. []
A) Duplicate B) NULL ☒ C) Unique D) Random0hi,
28. A table can have _____ primary key(s). []
A) Zero ☒ B) One C) Two D) Many
29. A foreign key is used to _____ two tables. []
A) Separate B) Join ☒ C) Link D) Delete
30. A foreign key references the _____ key of another table. []
A) Candidate B) Alternate ☒ C) Primary D) Composite

(Fill in the Blanks)

1. A database is an organized collection of Data that allows easy access, management, and updating.
2. DBMS stands for Database Management System.
3. The primary purpose of a DBMS is to store data and provide Efficient access to it.
4. Hierarchical, Network, and Relational are types of Database models.
5. A relational database stores data in the form of Tables.
6. The domain constraint specifies the Allowed values that an attribute can take.
7. A key constraint ensures that all values in a key attribute are Unique.
8. The NOT NULL constraint ensures that a column cannot have NULL values.
9. The SELECT operation in relational algebra is also known as the Selection operation.
10. A foreign key is used to establish a link between two Tables.
11. In a relational database, each row of a table is called a Tuple.
12. A non-relational database is also commonly known as a NoSQL database.
13. MongoDB is an example of a Non-Relational database.
14. One major advantage of the database approach over file systems is reduced data Redundancy.
15. Data independence means changes in the schema do not affect the Application programs.
16. The SQL command used to modify existing records is UPDATE.
17. The SQL command used to remove records from a table is DELETE.
18. The DROP statement is used to permanently delete a table from the database.
19. The command used to add a new column to an existing table is ALTER.
20. A primary key must contain Unique values and cannot be NULL.
21. A database schema represents the Structure of the database.
22. A diamond symbol in an ER diagram represents a Relationship.
23. A primary key is an attribute that uniquely identifies an Entity in an entity set.
24. Refinement of ER design involves removing redundancy and resolving Many-to-many relationships.
25. The main goal of database architecture is to achieve data Independence and security.
26. The INTERSECTION operation returns only the tuples that are Common in both relations.

27. In SQL, the command used to retrieve data from a table is SELECT.

28. The SQL command used to create a new table is CREATE TABLE.

29. The structure of a table is defined using the CREATE TABLE statement in SQL.

30. The primary purpose of constraints is to maintain data Integrity.